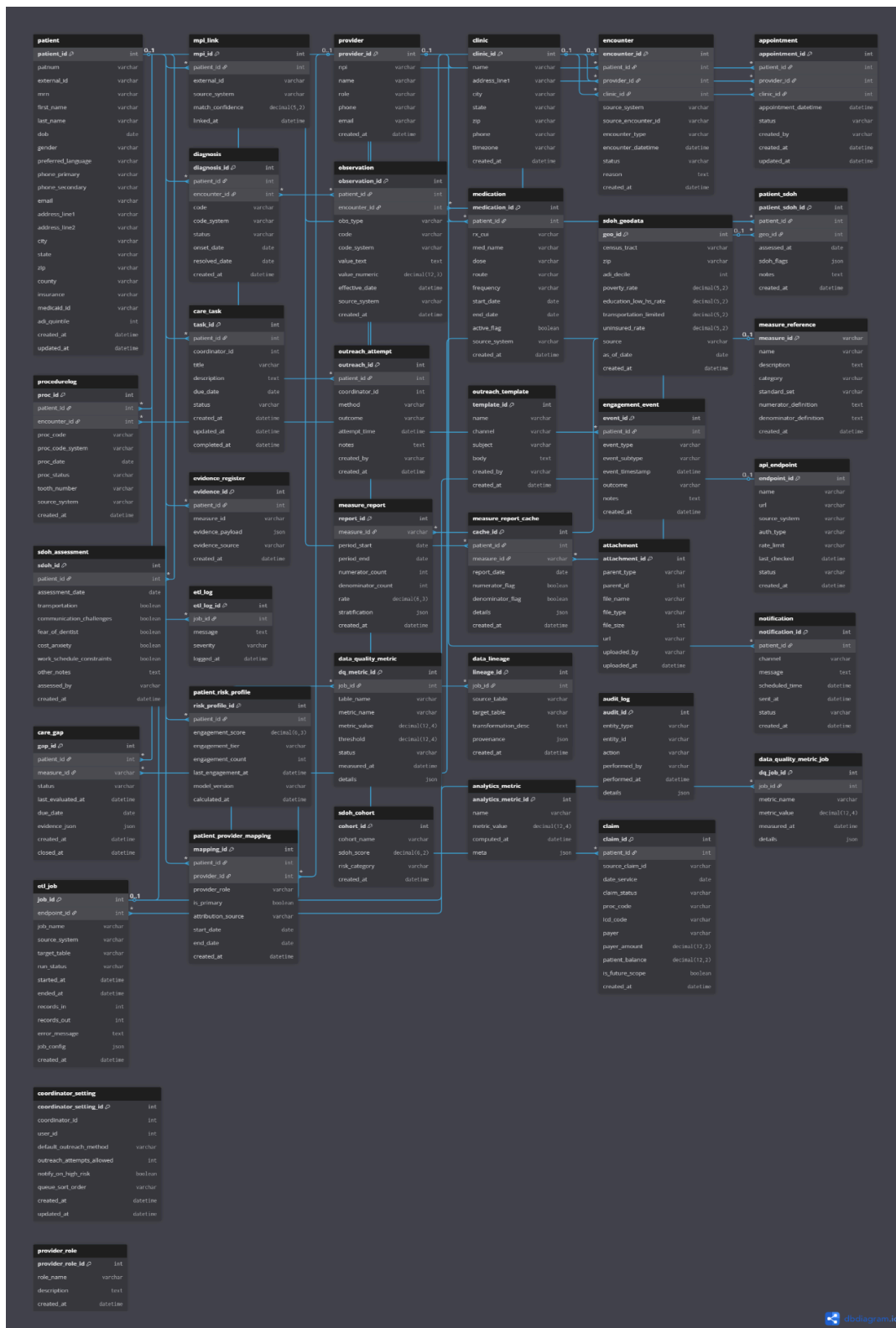


1.Canonical Data Model



1. Design Principles

The FulcrumCare canonical data model is designed to:

1. Unify medical, dental, and SDOH data at the patient level.
2. Preserve clinical source fidelity while enabling analytics.
3. Support care gaps, population health, and coordinated workflows.
4. Maintain auditability, lineage, and regulatory readiness.
5. Allow incremental extension (PCP, Dentist, Claims) without schema disruption.

The model follows a hub-and-spoke pattern, with **patient** as the central hub and domain-specific tables radiating outward.

2. Core Patient & Identity Layer

2.1 **patient**

Role

The **patient** table is the canonical patient record. Every downstream clinical, quality, and engagement artifact ultimately references this table.

Key Responsibilities

- Demographic identity
- Communication and address data
- Risk enrichment (ADI quintile)
- Temporal integrity (created/updated timestamps)

Relationships

- One-to-many with:
 - **encounter**
 - **diagnosis**
 - **observation**
 - **medication**
 - **care_gap**
 - **patient_sdo**

- `sdoh_assessment`
- `care_task`
- `outreach_attempt`
- `engagement_event`
- `patient_risk_profile`
- `claim`

This makes `patient` the primary aggregation anchor.

2.2 `mpi_link`

Role

`mpi_link` enables cross-system identity resolution (HIE MRN, OpenDental PatNum, payer IDs).

Why It Exists Separately

- Prevents contamination of the canonical patient row
- Supports probabilistic and evolving matches
- Enables re-linking without patient record mutation

Relationship

- Many MPI records → one patient
(`mpi_link.patient_id` → `patient.patient_id`)

3. Provider, Clinic & Attribution Layer

3.1 `provider`

Represents individual clinicians (PCPs, dentists, specialists).

Why Provider Is Decoupled

- Providers may exist without encounters (future attribution)
- Enables provider-centric analytics (quality ownership)

3.2 `clinic`

Represents physical or virtual care locations.

Key Role

- Encounter context
- Operational and geographic segmentation

3.3 `patient_provider_mapping` (Overlay Addition)

Purpose

This table models longitudinal provider attribution, not episodic care.

What It Solves

- PCP and dentist assignment outside encounters
- Attribution from multiple sources (HIE, dental EHR, payer)
- Historical tracking of changes over time

Relationship Pattern

- Many mappings per patient
- Many patients per provider
- Time-bound via `start_date` / `end_date`

This intentionally avoids altering `encounter.provider_id`.

4. Clinical Event Layer

4.1 `encounter`

Represents any clinical interaction (medical or dental).

Central Role

- Most clinical entities attach to encounters
- Preserves source encounter semantics

Key Relationships

- `patient` → `encounter`
- `provider` → `encounter`

- `clinic` → `encounter`

4.2 `appointment`

Represents scheduled intent, distinct from completed care.

Why Separate from Encounter

- Not all appointments result in encounters
- Supports no-show tracking and access analytics

4.3 `procedurelog`

Captures dental procedures (CDT-based).

Relationship

- Linked to both patient and encounter
- Supports tooth-level detail

5. Clinical Observations & Conditions

5.1 `diagnosis`

Stores problem lists and conditions.

5.2 `observation`

Stores measurements, lab results, vitals.

5.3 `medication`

Stores active and historical medications.

Design Choice

These tables are normalized and separated because:

- Each follows different coding systems
- They evolve independently
- They are used differently in measure logic

All link back to [patient](#), and where relevant, to [encounter](#).

6. SDOH & Community Context Layer

6.1 [sdoh_geodata](#)

Stores community-level indicators (ACS-derived).

Why It Is Not Patient-Owned

- Represents geographic facts, not patient facts
- Reused across many patients

6.2 [patient_sdoh](#)

Links patient → geodata snapshot.

Purpose

- Temporal capture of SDOH exposure
- Supports re-scoring when community data updates

6.3 [sdoh_assessment](#)

Captures directly assessed, patient-reported barriers.

Why Separate from [patient_sdoh](#)

- One is inferred/contextual
- One is explicitly assessed
- Different update cycles and trust levels

7. Care Coordination & Engagement

7.1 [care_task](#)

Represents actionable work items for coordinators.

7.2 [outreach_attempt](#)

Tracks individual outreach events.

7.3 outreach_template

Templates for standardized communication.

7.4 engagement_event

Tracks patient responses and engagement signals.

Relationship Pattern

- Tasks and outreach connect to patients
- Templates intentionally do not (they are reusable metadata)

8. Quality Measures & Care Gaps

8.1 measure_reference

Canonical definition of quality measures.

Why Standalone

- Measures are static, reusable logic entities
- Avoids duplication across reports

8.2 care_gap

Represents patient-measure state.

Core Function

- Open vs closed logic
- Due dates
- Evidence linkage

8.3 evidence_register

Stores defensible artifacts supporting gap status.

8.4 `measure_report` & `measure_report_cache`

- `measure_report`: aggregate performance
- `measure_report_cache`: patient-level flags for fast UI rendering

9. Risk, Cohorts & Analytics

9.1 `patient_risk_profile`

Stores derived risk scores.

Why Derived, Not Raw

- Model outputs must be auditable
- Model evolution requires versioning

9.2 `sdoh_cohort`

Represents population cohorts, not individuals.

Why No FK to Patient

- Cohorts are definitions
- Membership is resolved dynamically

9.3 `analytics_metric`

Generic metrics storage.

Why Detached

- Allows arbitrary analytics without schema churn

10. Claims (Future Scope)

10.1 `claim`

Designed as patient-centric, not encounter-centric.

Reason

- Claims often lack reliable encounter IDs
- Supports retrospective attribution

11. Observability, ETL & Governance

11.1 `api_endpoint`

Describes external data interfaces.

11.2 `etl_job`, `etl_log`

Execution and error tracking.

11.3 `data_quality_metric`

Quantitative data validation outputs.

11.4 `data_lineage`

End-to-end provenance and traceability.

Why These Are Lightly Related

- They operate at system level
- They should not block clinical transactions

12. Audit, Notifications & Attachments

12.1 `audit_log`

Intentionally polymorphic

Why No Foreign Keys

- Must log changes even when records are deleted
- Required for compliance integrity

12.2 `notification`

Links to patient but not specific workflows.

Why

- Notifications are delivery artifacts, not domain objects

12.3 attachment

Uses (parent_type, parent_id).

Reason

- Enables attachment reuse across many entities
- Avoids exponential FK growth

Population Health & Care-Gap Framework and the Risk stratification algorithm.

1 — OVERVIEW: calculation engine responsibilities

The Calculation Engine (CE) sits in the **Clinical Intelligence & Analytics** layer and is responsible for:

- Normalizing and validating inputs from the Unified Data Repository (UDR).
- Computing **care gaps** (per patient × measure) using CQL/FHIRPath logic.
- Generating **evidence packages** (JSON) per gap for audit and UI drilldown.
- Producing **measure reports** (numerator/denominator, measure rate).
- Computing **patient-level features** used in cohorts and risk-stratification.
- Generating **risk scores** where the primary driver is patient engagement (care coordinator activities), augmented by clinical and SDOH signals.
- Emitting audit logs, data lineage metadata, and data quality metrics.

All outputs must be traceable back to input sources (via evidence_register, lineage, audit_log, measure_report_cache).

2 — DATA INPUTS & OUTPUTS (canonical)

Inputs (tables in canonical model):

- `patient` (demographics)
- `encounter`, `diagnosis`, `procedure`, `observation`, `medication`
- `sdoh_geodata`, `patient_sdoh`, `sdoh_assessment`
- `evidence_register` (existing evidence)
- `patient_cohort` (cohort definitions)
- `engagement_event`, `outreach_attempt`, `care_task` (care coordinator activity)
- `measure_reference` (measure definitions, CQL/logic)
- `mpi_link`, `etl_job`, `data_quality_metric`, `data_lineage`, `audit_log` (operational metadata)

Primary Outputs:

- `care_gap` rows: {`patient_id`, `measure_id`, `gap_status`, `last_evaluated_at`, `evidence_id`}
- `evidence_register`: JSON payloads supporting gap decisions
- `measure_report` & `measure_report_cache`: patient-level and aggregated measure outcomes
- `patient_risk_profile`: `engagement_score`, `clinical_score`, `sdoh_score`, `composite_risk`, `risk_bucket`
- `analytics_metric` and `data_quality_metric` for observability

3 — CARE-GAP FRAMEWORK: design & algorithm

Design principles

- Use **canonical measure definitions** (`measure_reference`) expressed as **CQL/FHIRPath** where possible (HAPI CQL engine).
- Evaluate measures at **patient-level** for each reporting period.
- Support **open/closed gaps** and **evidence-driven overrides** (manual or external).
- Keep auditability: every gap decision writes an `evidence_register` JSON with provenance: `source_records`, `applied_rules`, `timestamp`, `job_id`.

Key concepts

- **Measure**: canonical ID, numerator/denominator logic, measurement period.
- **Gap**: for preventive/clinical measure, `gap_status` = OPEN | CLOSED | NOT_APPLICABLE.
- **Evidence**: JSON containing supporting resources (encounters, observation, proc, notes).

- **MeasureReport:** HAPI CQL output aggregated.

High-level flow

1. For each active `measure_reference` and each patient in scope, compute denominator inclusion.
2. If in denominator: evaluate numerator. If numerator satisfied → CLOSED, else → OPEN.
3. For OPEN gaps, produce `care_gap` with reason and evidence (most recent supporting negative evidence, e.g., last encounter beyond interval).
4. Persist evidence into `evidence_register` linked to the `care_gap`.
5. Update `measure_report` and `measure_report_cache`.
6. Emit lineage and audit logs.

Pseudocode (patient-level)

```
for each measure in active_measures:
    for each patient in target_population(measure):
        denom = evaluate_cql(measure.denominator, patient)
        if not denom:
            write_measure_report(patient, measure, status="N/A")
            continue

        num = evaluate_cql(measure.numerator, patient)
        now = current_timestamp()
        if num:
            gap_status = "CLOSED"
            evidence = collect_supporting_evidence(patient, measure, now,
satisfied=True)
        else:
            gap_status = "OPEN"
            evidence = collect_supporting_evidence(patient, measure, now,
satisfied=False)

        evidence_id = insert_evidence(evidence, patient_id, measure_id)
        insert_or_update_care_gap(patient_id, measure_id, gap_status,
evidence_id, now)
        update_measure_report(patient, measure, denom, num)
        register_audit_and_lineage(job_id, patient_id, measure_id,
source_ids)
```

Example CQL/SQL strategy (CQL preferred)

- Load the `measure_reference` CQL into HAPI CQL engine and call `evaluate(patient)` → returns boolean for numerator/denominator.
- If you cannot use CQL for PoC, express logic as parameterized SQL queries against `observation`, `procedure`, `diagnosis` tables.

Example SQL fragment (pseudo)

```
-- Determine if patient had preventive dental visit in last 12 months
SELECT 1
FROM procedure p
WHERE p.patient_id = :patient_id
      AND p.proc_code IN ('D0120','D0150') -- example CDT codes
      AND p.proc_date >= (CURRENT_DATE - INTERVAL '12 months')
LIMIT 1;
```

Evidence payload structure (JSON)

```
{
  "patient_id": 123,
  "measure_id": "MEASURE_XYZ",
  "evaluated_at": "2025-11-30T12:00:00Z",
  "result": "OPEN",
  "supporting_records": [

    {"table": "procedure", "id": 456, "source": "OpenDental", "date": "2024-10-20"},

    {"table": "observation", "id": 789, "source": "HIE", "date": "2023-09-01"}
  ],
  "explanation": "No preventive dental procedure in last 12 months",
  "computed_by": "care_gap_engine_v1",
  "job_id": "etl_job_20251130_01"
}
```

Notes on performance & scale

- Batch evaluation recommended: evaluate measures in cohorts using set-based SQL or CQL engine in bulk rather than patient-by-patient when possible.

- Cache `measure_report_cache` to serve dashboards and avoid re-evaluation on every UI request.

4 — RISK-STRATIFICATION ALGORITHM (engagement-driven)

***Note that this is a sample algorithm and the actual weighting will be determined by the clinical board**

You asked risk to be driven primarily by **engagement in the care coordinator flow** rather than simple numeric thresholds. Below is a formalized, configurable algorithm.

Design goals

- Risk is a **composite** of three weighted sub-scores:
 1. **Engagement Score (ES)** — primary driver: care coordinator interactions, outreach outcomes, task completion.
 2. **Clinical Score (CS)** — number/severity of open gaps, chronic condition burden, utilization (ED visits).
 3. **SDOH Score (SS)** — community-level ADI + coordinator-reported SDOH (transportation, cost anxiety, etc.)
- Composite score = $w1*ES + w2*CS + w3*SS$, with `w1` highest weight (e.g., 0.5), and `w2/w3` configurable.
- Risk buckets: LOW / MEDIUM / HIGH / VERY_HIGH (configurable thresholds).
- Scores are normalized [0..100].
- Use decay function on engagement events: older interactions count less.

Inputs (from canonical model)

- `engagement_event` (timestamp, event_type, outcome, coordinator_id)
- `outreach_attempt` (method, outcome, attempt_time)
- `care_gap` (open gap count, gap severity—mapped from measure importance)
- `sdoh_assessment` (binary flags: transportation, communication, cost_anxiety, dental_fear, work_schedule_constraints)
- `sdoh_geodata` (ADI decile, poverty_rate)
- `encounter` (ED visits count last 12 months), `claim` (future scope)

Scoring components definition-

1. Engagement Score (ES)

- For each patient, compute a weighted sum of recent engagement events, where:
 - `event_weight` depends on type: (call_answered=3, call_unanswered=1, message_response=2, missed_appt=2, task_completed=4, visit_scheduled=3)
 - `recency_decay = exp(-lambda * days_since_event)` with `lambda` chosen to reduce influence after e.g., 90 days.
- Normalize ES to 0..100 based on historical max or configured scaling.

Pseudocode:

```
ES_raw = sum_over_events(event_weight(event_type) *
  recency_decay(days_since_event) * outcome_multiplier)
ES = 100 * min(ES_raw / ES_scale, 1.0)
```

`outcome_multiplier` can boost successful engagement (e.g., answered=1.0, voicemails=0.3).

2. Clinical Score (CS)

- Compute from:
 - `open_gaps_count` weighted by gap importance from `measure_reference.category` (e.g., HEDIS high priority weight).
 - `ed_visits_last_12m` counted from `encounter` with `encounter_type='ED'`.
 - `chronic_condition_count` (diabetes, CHF, COPD, etc.)
- Use a weighted linear combination and normalize.

Pseudocode:

```
CS_raw = sum(open_gaps_i * weight_gap_i) + ed_visits*w_ed +
  chronic_count*w_chronic
CS = 100 * min(CS_raw / CS_scale, 1.0)
```

3. SDOH Score (SS)

- Combine census-derived ADI decile and coordinator-reported SDOH flags.

- ADI mapping: ADI decile 10 → ADI_component = 1.0, decile 1 → 0.0
- Coordinator flags: each critical flag adds incremental risk (transportation +0.25, communication +0.2, cost +0.3, dental_fear +0.1, work_schedule +0.15)
- Normalize to 0..100.

Pseudocode:

```
ADI_component = (ADI_decile - 1) / 9 # maps 1..10 to 0..1
sdoh_flag_sum = sum(flag_weights[flag] for flag in sdoh_flags_true)
SS_raw = ADI_component * w_adi + sdoh_flag_sum
SS = 100 * min(SS_raw / SS_scale, 1.0)
```

Composite & Bucketing

Set weights (configurable):

- w1 (engagement weight) default 0.5
- w2 (clinical weight) default 0.3
- w3 (sdoh weight) default 0.2

Compute:

```
composite_raw = w1*ES + w2*CS + w3*SS
composite = round(composite_raw,1) # 0..100
if composite >= 85: risk_bucket = "VERY_HIGH"
elif composite >= 70: risk_bucket = "HIGH"
elif composite >= 40: risk_bucket = "MEDIUM"
else: risk_bucket = "LOW"
```

Pseudocode: risk engine per patient

```
def compute_engagement_score(patient_id, now):
    events = fetch_engagement_events(patient_id)
    ES_raw = 0
    for ev in events:
        w = event_weight(ev.type)
        days = (now - ev.timestamp).days
        decay = exp(-lambda_decay * days)
        outcome_mult = 1.0 if ev.outcome == 'positive' else 0.4
        ES_raw += w * decay * outcome_mult
    ES = normalize(ES_raw, ES_scale)
```

```

    return ES

def compute_clinical_score(patient_id):
    open_gaps = fetch_open_gaps(patient_id) # returns list of
(measure_id, priority_weight)
    ed_visits = fetch_ed_visits_count(patient_id, last_12_months)
    chronic_count = fetch_chronic_condition_count(patient_id)
    CS_raw = sum(w for (_, w) in open_gaps) + ed_visits * w_ed +
chronic_count * w_chronic
    CS = normalize(CS_raw, CS_scale)
    return CS

def compute_sdo_h_score(patient_id):
    adi = fetch_adi_decile(patient_id)
    flags = fetch_sdo_h_flags(patient_id)
    adi_comp = (adi - 1)/9
    sdo_h_sum = sum(flag_weights[f] for f in flags if flags[f])
    SS_raw = adi_comp * w_adi + sdo_h_sum
    SS = normalize(SS_raw, SS_scale)
    return SS

def compute_composite(patient_id):
    ES = compute_engagement_score(patient_id, now)
    CS = compute_clinical_score(patient_id)
    SS = compute_sdo_h_score(patient_id)
    composite = w1*ES + w2*CS + w3*SS
    bucket = bucketize(composite)
    persist_patient_risk_profile(patient_id, ES, CS, SS, composite,
bucket, now)
    emit_analytics_metric(patient_id, composite)

```

Example normalized numeric mapping (recommended defaults)

- `ES_scale = 30` (so `ES_raw/30` → fraction)
- `CS_scale = 20`
- `SS_scale = 2.0`
- `event_weight`: `call_answered=4`, `message_response=3`, `task_completed=5`,
`missed_appt=2`, `voicemails=1`

- **flag_weights**: transportation=0.3, communication=0.25, cost=0.35, dental_fear=0.15, work_schedule=0.2
- **lambda_decay** = $\ln(2)/30$ so half-life ~30 days

Adjust scales after PoC based on distribution.

5 — SAMPLE SQL/PSEUDO-IMPLEMENTATION

Engagement raw score via SQL (simplified)

```
WITH events AS (
  SELECT ev.patient_id,
    SUM(
      CASE ev.event_type
        WHEN 'task_completed' THEN 5
        WHEN 'call_answered' THEN 4
        WHEN 'message_response' THEN 3
        WHEN 'missed_appointment' THEN 2
        ELSE 1
      END * EXP(-:lambda * (DATE_PART('day', now()) -
ev.event_timestamp)))
    ) AS eng_raw
  FROM engagement_event ev
  WHERE ev.patient_id = :patient_id
  GROUP BY ev.patient_id
)
SELECT patient_id,
  LEAST(100, 100 * (eng_raw / :es_scale)) AS engagement_score
FROM events;
```

Open gaps count with weights SQL

```
SELECT p.patient_id,
  SUM(m.priority_weight) AS open_gap_weight_sum
FROM care_gap cg
JOIN measure_reference m ON cg.measure_id = m.measure_id
JOIN patient p ON cg.patient_id = p.patient_id
WHERE cg.gap_status = 'OPEN' AND p.patient_id = :patient_id
GROUP BY p.patient_id;
```

SDOH score

```
SELECT p.patient_id,  
       ((sdoh.adi_decile - 1)/9.0) * :w_adi  
       + (CASE WHEN s.transportation THEN :w_transport ELSE 0 END)  
       + (CASE WHEN s.communication_challenges THEN :w_comm ELSE 0  
END)  
       + (CASE WHEN s.cost_anxiety THEN :w_cost ELSE 0 END) AS  
sdoh_raw  
FROM patient p  
LEFT JOIN patient_sdoh s ON p.patient_id = s.patient_id  
LEFT JOIN sdoh_geodata sdoh ON s.geo_id = sdoh.geo_id  
WHERE p.patient_id = :patient_id;
```

6 — EVIDENCE & AUDITABILITY (must-have)

- Every `care_gap` decision writes `evidence_register` entry with:
 - `supporting_records` (ids and source_system)
 - `logic_version` and `measure_id`
 - `job_id`
- `audit_log` entry for the job records user/system actor, start/stop times, and counts processed.
- `data_lineage` entry mapping the source tables used for each gap.

This enables audit by measure, patient, or time window—key for SOC2/HIPAA.

7 — OBSERVABILITY & METRICS

Emit the following per job run:

- `etl_job` entries with counts_in, counts_out, errors_count, duration.
- `data_quality_metric`: percent completeness of required fields, duplicate rate, percent invalid codes.
- `analytics_metric`: number of gaps computed, risk distribution (count per bucket), average composite score.
- Alerting triggers:
 - sudden drop in engagement_event ingests
 - metrics variation > 30% week-over-week
 - data quality threshold breaches (e.g., >5% invalid codes)

8 — DEPLOYMENT & CONFIGURATION

1. Deployment Overview

The Fulcrum platform will be deployed as a **modular, containerized, service-oriented system**, where each architectural block maps cleanly to independently deployable components. The deployment emphasizes:

- Clear separation of concerns (ingestion, analytics, coordination, visualization)
- Horizontal scalability for ETL and analytics workloads
- Strong observability, auditability, and compliance
- Minimal coupling between services, with PostgreSQL acting as the central system of record

The platform will be deployed on a **container orchestration environment** (Docker with Kubernetes or equivalent), with managed PostgreSQL instances and secure API-based communication between services.

2. Data Sources Layer (External Systems)

Components

- ACS SDOH APIs
- FHIR HIE / QHIN (FHIR R4 APIs)
- OpenDental EHR
- Claims Data (Future Scope)

Deployment Characteristics

- These systems are **external** and **not deployed by Fulcrum**
- Secure outbound connectivity is configured from the ingestion layer
- Authentication handled via:
 - OAuth2 / SMART-on-FHIR (FHIR HIE)
 - API keys / credentials (ACS, OpenDental)
- Network access restricted using IP allowlists and TLS

No persistent infrastructure is required in this layer within the Fulcrum environment.

3. FHIR Gateway Layer (Dockerized Service)

Technology

- **FHIR Gateway:** HAPI FHIR
- **Deployment:** Docker container
- **Auth:** SMART on FHIR / OAuth2

Deployment Details

- Deployed as a **standalone Docker service**
- Only processes:
 - FHIR HIE / QHIN data
- Exposes internal REST endpoints to ETL Orchestration
- Stateless service:
 - Horizontal scaling supported
 - No persistent storage
- Configuration via environment variables:
 - OAuth credentials
 - FHIR endpoint URLs
 - Token refresh policies

Key Responsibility

- Normalize incoming FHIR R4 resources
- Validate conformance
- Act as the **FHIR-only ingestion boundary**

4. ETL Orchestration Layer (Apache Airflow)

Technology

- **Apache Airflow**
- **Executors:** KubernetesExecutor or CeleryExecutor

Deployment Details

- Deployed as a **dedicated Airflow cluster**

- Components:
 - Scheduler
 - Webserver
 - Worker nodes
- DAGs configured for:
 - ACS ingestion
 - OpenDental ingestion
 - FHIR Gateway ingestion
 - (Future) Claims ingestion

Execution Characteristics

- Stateless workers with autoscaling
- All intermediate results written to PostgreSQL
- Retries, backfills, and failure recovery handled by Airflow

Integration

- Calls FastAPI services (MPI, Validation) via REST
- Writes validated data to Unified Data Repository

5. MPI, Validation & Mapping Services (FastAPI)

Technology

- **FastAPI**
- **Deployment:** Docker containers

Services Deployed

- Master Patient Index (MPI)
- Validation Engine
- Mapping & Canonicalization Service

Deployment Details

- Each service deployed independently
- Stateless APIs
- Horizontally scalable behind a load balancer

- Connected to PostgreSQL for:
 - Identity resolution
 - Match confidence tracking
 - Canonical record persistence

Responsibilities

- Cross-source patient identity resolution
- Coding validation (ICD, CDT, SNOMED)
- Schema enforcement
- Canonical data generation

6. Unified Data Repository & Operational Data Store (PostgreSQL)

Technology

- **PostgreSQL**
- **Deployment:** Managed PostgreSQL (recommended)

Databases

- Unified Clinical Data Store
- Care Coordination Operational Store

Deployment Characteristics

- Single logical cluster with schema separation
- ACID-compliant transactions
- Read replicas for analytics workloads
- Encrypted at rest and in transit

Stored Data Includes

- Canonical patient records
- Medical, dental, and SDOH data
- Engagement and outreach data
- Analytics outputs and aggregates

This layer acts as the **authoritative system of record**.

7. Observability, Monitoring & Compliance Layer

Technology

- **Pandas** (data profiling, validation metrics)
- **SigNoz** (metrics, traces, logs)

Deployment Details

- SigNoz deployed as:
 - Collector
 - Query service
 - UI
- Instrumentation enabled across:
 - Airflow
 - FastAPI services
 - Database interactions

Observability Coverage

- ETL job execution
- API latency and errors
- Data quality metrics
- Audit logs
- Lineage metadata

Compliance

- HIPAA-aligned audit trails
- Immutable logs
- Access tracking

8. Analytics Layer (FastAPI)

Technology

- **FastAPI**
- **Python analytics stack**

Services Deployed

- Care Gap Engine
- Measure Calculation Engine
- Population Health Analytics
- Risk Stratification Engine

Deployment Details

- Stateless FastAPI services
- Scaled independently based on workload
- Heavy computation handled asynchronously
- Results persisted to PostgreSQL

Data Inputs

- Unified Data Repository
- Care Coordination Operational Data

Outputs

- Measure results
- Risk scores
- Cohort assignments
- Analytics summaries

9. Dashboards & Care Coordination Layer

Technology

- **Frontend:** React
- **Backend APIs:** FastAPI
- **Data Store:** PostgreSQL

Deployment Details

- React application deployed as:
 - Static web app

- Served via CDN or web server
- Backend APIs deployed as FastAPI services
- Secure authentication and role-based access

Functional Areas

- Clinical dashboards
- Population views
- Care coordinator workflows
- Engagement tracking

10. Care Coordination Operational Data Layer

Technology

- PostgreSQL

Deployment Details

- Separate schema or logical database
- Direct writes from care coordinator workflows
- Integrated into analytics and risk stratification

Stored Data

- Outreach attempts
- Engagement status
- SDOH assessments
- Notes and outcomes

This data **feeds back** into analytics and risk scoring.

9 — EXAMPLE: end-to-end for a single measure

1. Job picks **MEASURE_DENTAL_PREV_12M**.
2. For cohort patients, runs CQL denominator: age and active patient.
3. For those in denominator, runs CQL numerator: any procedure code in last 12 months.
4. For patients with no numerator hit:
 - create **care_gap** OPEN
 - prepare evidence JSON with last dental encounter and last attempt outreach events

- write `evidence_register` and `audit_log` and `data_lineage` entries
5. Summarize results into `measure_report_cache` for dashboard consumption.

10 — Governance & Privacy considerations

- All CE infrastructure must respect PHI security: run in VPC/private subnets, encrypt-at-rest (Postgres, Mongo), TLS in transit.
- Limit access to `evidence_register` and `audit_log` to authorized roles.
- Maintain retention policies for evidence (configurable), and data minimization for analytics exports.